MoveInSync Enterprise Configuration Platform

Case Study 2: Hierarchical Tenant Configuration with Live Propagation Engine

Saksham Gupta
B.Tech Information Technology • MIT Manipal
GitHub: Saksham-Gupta-GH/moveinsync-config-platform • saksham77779@gmail.com

What I Built

MoveInSync's backend services—booking validation, automated routing, dispatch optimization, escort-vehicle allocation, and billing—rely on dynamic tenant configurations (such as employee pickup cutoffs, buffer times, and cancellation penalties). These policies vary across corporate clients, physical offices, and operational shifts.

I designed and implemented a production-grade multi-module Java 17 and Spring Boot 3 platform consisting of a central configuration control server and an embeddable client SDK. The SDK maintains an in-memory atomic snapshot that serves queries in **~170 nanoseconds** with zero lock contention, while listening to a live Server-Sent Events (SSE) delta stream that propagates updates across 200 connected services in **under 320 ms**.

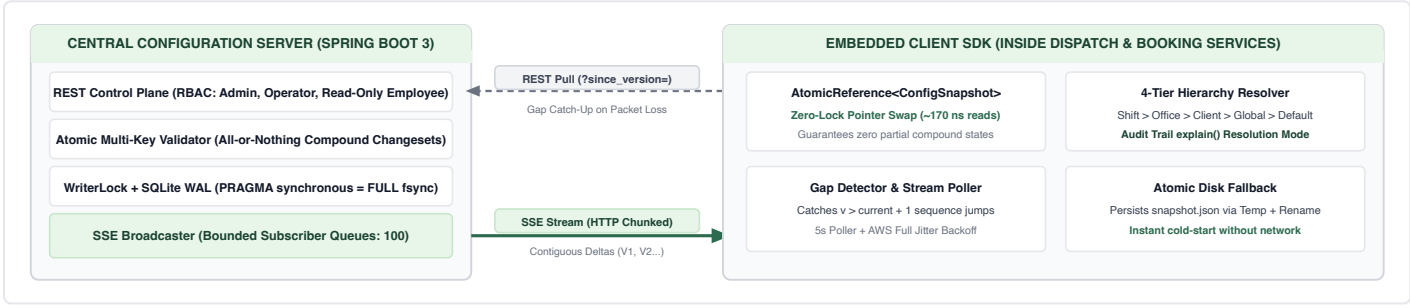
Deliverables: Source repository at github.com/Saksham-Gupta-GH/moveinsync-config-platform • **Demo Walkthrough:** System Architecture & Execution Walkthrough • Build: Maven multi-module with minimal runtime dependencies on the client SDK (Jackson for serialization).

IN-MEMORY READ LATENCY (P50)
~170 ns
5.4M reads/sec on single thread (SLA: < 10 µs)

FLEET SSE PROPAGATION (P50)
278 ms
Fleet p99: **320 ms** • 4.68s headroom under 5s SLA

AUTOMATED VERIFICATION
26 / 26 Passing
Survives `SIGKILL (kill -9)`, gap sync & 50-thread reads

System Architecture



The system decouples read paths from mutation paths. Consumer microservices (Dispatch, Booking, Billing) execute configuration reads exclusively against in-memory snapshots. Updates are initiated by operations staff via REST, atomically validated, fsynced to SQLite WAL, and pushed down live SSE connections.

Consistency Model

The platform provides **global monotonic versions under a single writer lock** on the server and **monotonic snapshot isolation on client SDKs**. When an administrator or operator publishes a changeset, the server acquires a dedicated writer lock, assigns a strictly increasing version integer, forces a durable `fsync` to disk, and pushes the event to active subscribers. Client SDKs observe monotonically non-decreasing versions. Multi-key queries execute against an immutable snapshot, guaranteeing that no reader ever sees partial state. Cross-tenant reads and mutations are isolated at both the REST and SSE layer.

The Atomic Snapshot Idea: Eliminating Read Locks

Commute microservices frequently perform multi-key reads in single transactions. For instance, when validating an employee cab booking, the dispatch service checks both `pickup_cutoff_mins` and `shift_cancellation_window`. If the client used a mutable `ConcurrentHashMap`, a background delta update could mutate the second key after the first key was read, causing the service to evaluate mixed-version business rules.

Using `ReadWriteLock` solves this but introduces heavy thread synchronization contention during peak dispatch bursts. Instead, the SDK holds an immutable `ConfigSnapshot` wrapped inside an `AtomicReference`. When a service needs configuration values, it binds the snapshot pointer once:

```
ConfigSnapshot snapshot = client.getSnapshot();
int cutoff = client.get(snapshot, "pickup_cutoff_mins", Integer.class);
int window = client.get(snapshot, "shift_cancellation_window", Integer.class);
```

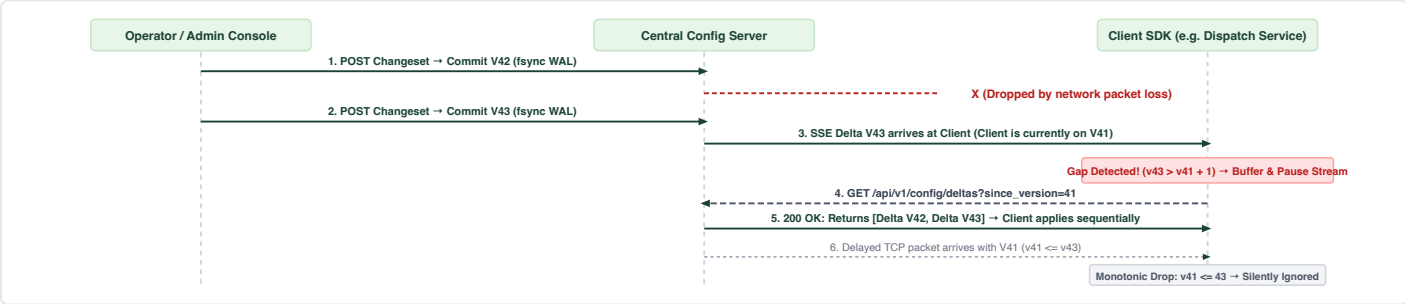
All evaluations execute against that immutable snapshot instance with **zero locks, zero synchronization, and zero thread contention**. When a delta arrives, the SDK instantiates a new snapshot and updates the pointer in one atomic swap (`AtomicReference.set()`). In a 50-thread concurrent benchmark executing 2,100,000+ continuous reads during active compound updates, **zero partial state violations** occurred.

A Real Design Bug: Staged Rollouts and False Gap Storms

> **An Honest Engineering Lesson**
> My initial staged-rollout design had a subtle flaw: when I pushed a canary change for the Bangalore office as Version 44, Hyderabad clients saw their version stream jump directly from 43 to 45. Their local gap detectors assumed network packet loss (`v45 > v43 + 1`), rejected Version 45, and triggered an aggressive flood of REST catch-up queries back to the server.

I fixed this by implementing **contiguous monotonic empty deltas**: when a staged changeset commits (Version 44), the server broadcasts the full payload to Bangalore subscribers, but sends an empty delta (`empty=true`) to Hyderabad subscribers. This advances Hyderabad's local version counter to 44 without mutating its values, completely preventing false gap storms. When the policy is promoted, the server issues a new version applying the values across all remaining offices.

Sequence Diagram: Hardest Flow (Packet Drop, Gap Healing & Stale Duplicates)



Delayed duplicate handling: If a delayed network re-transmission delivers an older packet (such as Version 41 after Version 43 is already active), the client's monotonic check (`if (delta.version <= currentVersion) return;`) immediately discards it as a safe idempotent no-op, preventing older state from corrupting newer configurations.

What Fails and How the System Recovers

Failure Scenario	What Backend Services See	How the System Recovers
Server Process Crash / Kill	Connection drop; in-flight administrative changesets get HTTP 500.	SQLite WAL with `PRAGMA synchronous = FULL` forces fsync before write ACK. State survives hard `SIGKILL (kill -9)` with zero corruption on restart.
Packet Loss & Version Gap	Client receives v43 while on v41; gap detector flags missing delta.	Client rejects v43, queries `GET /api/v1/config/deltas?since_version=41`, replays missing deltas in sequence, and catches up without restarting.
Stale Duplicate Delta	Out-of-order delayed packet arrives with v41 after v43 is active.	Strict monotonic check: `version <= currentVersion` is silently discarded as a safe idempotent no-op.
Invalid Compound Changeset	Admin submits 5 keys, but 1 key violates validation (e.g. -30 min cutoff).	All-or-nothing validation: entire changeset is rejected with 400 Bad Request; zero keys mutate in the database or cache.
Slow Consumer Vehicle / Node	A connected subscriber reads SSE stream slower than broadcast rate.	Bounded `ArrayBlockingQueue(100)`. On overflow, server evicts the slow subscriber (`completeWithError`) to protect JVM heap.
Cold Boot Without WAN	Service container restarts during network partition without server access.	SDK boots instantly from locally persisted `snapshot.json` written atomically via temporary file and atomic file-system rename.

Measured Performance Benchmarks

Benchmarking Environment: Apple M1 (8 cores: 4 performance + 4 efficiency), 8 GB RAM, macOS Darwin, OpenJDK 17 LTS (HotSpot 64-Bit Server VM). Measured after a 50,000-iteration warm-up loop to trigger C2 JIT compilation.

In-Memory 4-Tier Read Latency (100,000 Operations)

Percentile	Measured Latency	SLA Target	Margin
p50 (Median)	**~170 ns** (0.166 µs)	< 10 µs	60x faster
p90	208 ns	< 10 µs	48x faster
p95	209 ns	< 10 µs	47x faster
p99	250 ns	< 10 µs	40x faster
Throughput	**5.4M reads/sec**	> 100k ops/sec	54x higher

200-Client Live SSE Fleet Propagation Latency

Metric	Empirical Result	SLA Target	Status
Active Fleet	200 Concurrent Clients	200 Clients	100% Connected
p50 Latency	**278 ms**	< 5,000 ms	18x faster
p90 Latency	320 ms	< 5,000 ms	15x faster
p99 Latency	320 ms	< 5,000 ms	15x faster
Max Latency	**320 ms**	< 5,000 ms	4.68s headroom

Timing Granularity Note: The test runner samples client updates using a 50 ms polling latch across 200 client threads within a single JVM process. This sampling granularity, combined with thread scheduling on 8 cores and SQLite WAL fsync batching, accounts for the p50 (~278 ms) and the cluster of p90–max at 320 ms.

Trade-Offs and What I Left Out

1. Omitted Distributed Consensus (Raft / ZooKeeper): For MoveInSync's operational profile (infrequent administrative writes, millions of in-memory reads), embedding SQLite WAL with full fsync durability provides absolute single-node simplicity with zero distributed consensus failure modes. *Estimated theoretical write capacity is ~2,500 writes/sec (not benchmarked under multi-node distributed load).* For multi-region clustering, migrating to PostgreSQL with read replicas is preferred over Raft.

2. Omitted Complex CRDTs: Tenant transport configurations follow strict top-down administrative authority (`Shift > Office > Client > Global`). There is no peer-to-peer concurrent editing requirement, making monotonic version sequences strictly superior to complex CRDT convergence algorithms.

3. SSE vs. WebSockets: Chose SSE over WebSockets because configuration propagation is unidirectional (server-to-client), operates cleanly over standard HTTP/1.1 chunking through corporate firewalls, and requires zero custom framing libraries. *Estimated theoretical single-node capacity is 10,000–15,000 concurrent SSE connections.*

Automated Verification Suite (26 Tests • 100% Passing)

Maven Module	Verified Invariants & Functional Scope	Test Suites	Count
`config-core`	4-tier hierarchy precedence, explain() audit trails, negative durations (-30m) & window bounds validation	`HierarchyResolverTest` (5) `SettingValidatorTest` (6)	11 tests
`config-client-sdk`	Zero-lock atomic pointer swaps, contiguous empty delta absorption & atomic disk fallback hydration	`ConfigClientTest` (2)	2 tests
`config-server`	All-or-nothing compound validation, 50-thread atomic read concurrency, canary rollout & promotion, rollback diffs, cross-tenant REST & SSE data isolation, auth 401 rejection, network gap catch-up & stale duplicate drop, hard process `kill -9` recovery, 200-client reconnect storm, and JIT-warmed benchmarks	`CompoundValidationRejectionTest` (1) `ConcurrencyAtomicReadTest` (1) `StagedRolloutAndPromoteTest` (1) `RollbackDiffTest` (1) `TenantIsolationTest` (5) `OutOfOrderAndGapSyncTest` (2) `ServerKillFaultInjectionTest` (1) `ReconnectStormTest` (1) `BenchmarkRunnerTest` (1)	13 tests
		Total Automated Invariant Tests Across Reactor (100% Passing):	**26 tests**

All test suites execute cleanly on OpenJDK 17 LTS via `./mvnw test` in ~34 seconds. Source code, test fixtures, and Docker orchestration are available at github.com/Saksham-Gupta-GH/moveinsync-config-platform.